A mixed-base encoding and FAR-style invariant search for Antihydra

Abstract

Antihydra is the BB(6) candidate `1RB1RA_0LC1LE_1LD1LC_1LA0LB_1LF1RE_--0RA`. We give a self-contained account of its integer dynamics and of the mixed binary–ternary rewriting system used in the FAR/CPS search. The rewriting system is derived from the macro-level analysis of the underlying Turing machine; its transport rules preserve value and counter, and every rewriting strategy simulates the integer orbit step-for-step.

1 From the Turing machine to integer dynamics

1.1 The Antihydra Turing machine and its macro rules

Antihydra is the 6-state Turing machine with the following transition table. The tape alphabet is $\{0, 1\}$ and the states are $\{A, B, C, D, E, F\}$; the initial state is A and the initial tape is blank.

state	read 0	read 1
A	$1RB$	$1RA$
B	$0LC$	$1LE$
C	$1LD$	$1LC$
D	$1LA$	$0LB$
E	$1LF$	$1RE$
F	halt	$0RA$

For the analysis it is convenient to work with a macro-level configuration. Following the Lean formalisation in `bbchallenge/Antihydra/Antihydra.lean`, for $b, m, n, p \in \mathbb{N}$ define

$$P(b, m, n, p) = \big(\text{state } E,\ \text{head } 0,\ \text{left} = 1^m\, 0\, 1^b,\ \text{right} = 1^n\, 0^p\big). \tag{1}$$

The block 1^m is the "value block", the block 1^b is a unary counter, and the right block 1^n is used during the macro step.

The machine-checked macro theorems proved in `Antihydra.lean` are the following. (We write $C \xrightarrow{t} D$ to mean that the configuration C reaches D after exactly t steps.)

Lemma 1.1 (Macro loop step)**.** For all $b, m, n, p \in \mathbb{N}$,

$$P\big(b, m+4, n, p+2\big) \xrightarrow{2n+12} P\big(b, m+2, n+3, p+1\big).$$

Lemma 1.2 (Macro loop, k iterations)**.** For all $b, m, n, p, k \in \mathbb{N}$,

$$P\big(b, m+2+2k, n, p+1+k\big) \xrightarrow{k(2n+3k+9)} P\big(b, m+2, n+3k, p+1\big).$$

Lemma 1.3 (Even endgame)**.** For all $b, N, p \in \mathbb{N}$, the configuration $P(b, 0, N, p+2)$ halts in exactly 2 steps.

Lemma 1.4 (Odd endgame, counter positive)**.** For all $b', N, p \in \mathbb{N}$,

$$P\big(b'+1, 3, N, p+2\big) \xrightarrow{3N+20} P\big(b', N+6, 0, p\big).$$

Lemma 1.5 (Even endgame to next loop)**.** For all $b, N, p \in \mathbb{N}$,

$$P\big(b, 2, N, p+2\big) \xrightarrow{3N+2b+26} P\big(b+2, N+5, 0, p\big).$$

Lemma 1.6 (Odd halt endgame)**.** For all $N, p \in \mathbb{N}$, the configuration $P(0, 3, N, p+2)$ halts in exactly $2N + 12$ steps.

These six facts are enough to reduce the behaviour of the machine, once it has entered the macro regime, to a discrete dynamical system on a pair of nonnegative integers.

1.2 The integer recurrence

Projecting the macro transitions onto the appropriate integer value coordinate gives the following familiar Collatz-like map. Write the integer value as $a \in \mathbb{N}$ and the counter as $b \in \mathbb{N}$. The macro analysis implies that, as long as $a \geq 2$, the next pair is

$$(a, b) \longmapsto \begin{cases} \big(\lfloor 3a/2 \rfloor,\ b+2\big), & a \text{ even}, \\ \big(\lfloor 3a/2 \rfloor,\ b-1\big), & a \text{ odd and } b > 0, \\ \text{halt}, & a \text{ odd and } b = 0. \end{cases} \tag{2}$$

Equivalently, writing $a = 2n$ when a is even and $a = 2n+1$ when a is odd,

$$(2n, b) \mapsto (3n, b+2), \qquad (2n+1, b) \mapsto \begin{cases} (3n+1, b-1), & b > 0, \\ \text{halt}, & b = 0. \end{cases} \tag{3}$$

The machine halts precisely when an odd value is encountered while the counter is empty. Antihydra is believed to run forever, because the walk on the counter has positive drift ($+2$ with probability $1/2$, -1 with probability $1/2$).

The initial macro configuration reached after the 58-step warm-up is $P(2, 8, 0, 0)$. In the mixed-base encoding introduced below this configuration is written $\langle fff \rangle$ and has value $a_0 = 8$ and counter $b_0 = 0$.

Remark on the value coordinate. In `Antihydra.lean` the abstract `MathState` tracks the length m of the left block 1^m in (1); its even branch reads $2n \mapsto 3n+2$. The mixed-base encoding below uses a different Turing-machine invariant, namely the value $\mathrm{Val}(< w >)$ of the digit string, which satisfies the cleaner Mahler form (2). Both invariants give the same halting condition, and the Python simulation in `antihydra_far.py` validates (2) against the actual Turing-machine orbit for hundreds of dynamic steps.

2 Mixed-base rewriting rules

2.1 Alphabet and value function

The mixed binary–ternary encoding uses the alphabet

$$\Sigma = \{\ <,\ >,\ f,\ t,\ 0,\ 1,\ 2,\ c\ \}.$$

The symbols f and t are binary digits (f for $2x$, t for $2x+1$); the symbols $0, 1, 2$ are ternary digits ($3x$, $3x+1$, $3x+2$). The symbol $<$ is the left boundary, $>$ is the right boundary of the value word, and c is a unary counter symbol.

A *configuration* is a string of the form

$$< w > c^b, \qquad w \in \{f, t, 0, 1, 2\}^*,\ b \in \mathbb{N}. \tag{4}$$

The value of the configuration is computed by reading the value word $< w >$ from left to right:

$$\mathrm{Val}(< w > c^b) = \mathrm{Val}_<(w), \tag{5}$$

where the partial function $\mathrm{Val}_x(u)$ is defined recursively by

$$\begin{aligned} \mathrm{Val}_x(\varepsilon) &= x, \\ \mathrm{Val}_x(f\,u) &= \mathrm{Val}_{2x}(u), \\ \mathrm{Val}_x(t\,u) &= \mathrm{Val}_{2x+1}(u), \\ \mathrm{Val}_x(d\,u) &= \mathrm{Val}_{3x+d}(u) \qquad (d \in \{0,1,2\}), \\ \mathrm{Val}_x(>\ u) &= x. \end{aligned} \tag{6}$$

Thus $<$ contributes the initial value 1, each subsequent digit applies the corresponding affine function, and $>$ returns the current value. The counter symbols c do not affect the value.

Example 2.1. The initial configuration is $< fff >$. Its value is

$$\mathrm{Val}(< fff >) = 1 \xrightarrow{f} 2 \xrightarrow{f} 4 \xrightarrow{f} 8.$$

More generally, a binary word w ending in f or t represents the integer whose binary expansion is the sequence of bits encoded by w.

2.2 The rewriting system

The rewriting relation is generated by the following rules. The rules are divided into three groups: dynamic rules, which implement the integer map and change the counter; transport rules, which are value-preserving and move digits; and boundary rules, which turn a ternary digit at the left end back into binary digits.

Dynamic rules.

$$\begin{aligned} f > \ &\longrightarrow\ 0 > cc, && \text{(even value: } 2n \mapsto 3n,\ b \mapsto b+2), \\ t > c \ &\longrightarrow\ 1 >, && \text{(odd value: } 2n+1 \mapsto 3n+1,\ b \mapsto b-1). \end{aligned} \tag{7}$$

The second rule is applicable only when a counter symbol c is present; if the configuration ends in $t >$ with no c then no rule applies and the configuration is halting.

Transport rules.

$$\begin{aligned} &f0 \to 0f, \quad f1 \to 0t, \quad f2 \to 1f, \\ &t0 \to 1t, \quad t1 \to 2f, \quad t2 \to 2t. \end{aligned} \tag{8}$$

Each of these rules preserves the represented value: for example, $f0(x) = 2{\cdot}(3x) = 6x = 0f(x)$ and $t1(x) = 3 \cdot (2x+1) + 1 = 6x + 4 = 2f(x)$.

Boundary rules.

$$< 0 \to < t, \qquad < 1 \to < ff, \qquad < 2 \to < ft. \tag{9}$$

These implement the equation $3 = 2 \cdot 1 + 1$, $4 = 2 \cdot (2 \cdot 1)$, and $5 = 2 \cdot (2 \cdot 1) + 1$ at the left boundary.

The full set of rules is therefore

$$\begin{aligned} \mathcal{R} = {} & \{f > \to 0 > cc,\ t > c \to 1 >\} \\ & \cup \{f0 \to 0f,\ f1 \to 0t,\ f2 \to 1f,\ t0 \to 1t,\ t1 \to 2f,\ t2 \to 2t\} \\ & \cup \{< 0 \to < t,\ < 1 \to < ff,\ < 2 \to < ft\}. \end{aligned} \tag{10}$$

Definition 2.2 (Initial and halting configurations)**.** The initial configuration is $< fff >$, which has value $a_0 = 8$ and counter $b_0 = 0$. A configuration is *halting* if it ends in $t >$ and contains no counter symbol, i.e. it has the form $u\,t >$ with $u \in \Sigma^*$ and no c occurs.

2.3 Simulation

The rewriting system is designed so that the rightmost-rule strategy simulates the integer recurrence (2). More precisely, define a *dynamic firing* to be an application of one of the two dynamic rules. Then:

- Every transport or boundary rule preserves both the value $\mathrm{Val}(\cdot)$ and the number of counter symbols.
- A dynamic firing $f > \to 0 > cc$ applies exactly when the current value a is even, and it changes the value to $\lfloor 3a/2 \rfloor$ while adding two counter symbols.
- A dynamic firing $t > c \to 1 >$ applies exactly when the current value a is odd and the counter is positive, and it changes the value to $\lfloor 3a/2 \rfloor$ while removing one counter symbol.
- If the value is odd and the counter is empty, the configuration ends in $t >$ and no rule applies: this is the halting condition.

A useful structural fact, inherited from the YAH framework, is that the counter available at the j-th odd step is independent of the rewriting strategy: transport and boundary rules preserve the value and the counter, and dynamic steps depend only on the parity of the current value. This is the analogue for Antihydra of the strategy-independence argument in the proof of Theorem 3.17, Claim 1 of Yolcu–Aaronson–Heule ([1, pp. 20–21]): there the auxiliary rules $X = A \cup B$ preserve $\mathrm{Val}(\cdot)$ while the dynamic rules DT apply the Collatz map T to the value. Hence the nondeterministic closure of the rewriting system decides exactly the same halting question as the deterministic integer orbit.

Example 2.3. The first few dynamic steps from the initial configuration are:

$$< fff > \xrightarrow{f > \to 0 > cc} < ff0 > cc \xrightarrow{\text{transport}} \cdots \xrightarrow{f > \to 0 > cc} < 0 \ldots 0 > c^b.$$

At each dynamic firing the represented value follows $8, 12, 18, 27, \ldots$, and the counter follows $0, 2, 4, 6, \ldots$, exactly as prescribed by (2).

3 Formal verification in Lean

The claims of the preceding sections have been formalised in Lean 4 in the directory `lean-code/SRS`. The relevant files are `Basic.lean` (string-rewriting systems), `MixedBaseRepresentation.lean` (the YAH mixed-base value function), `AntihydraSRSaxiom.lean` (a BusyLean-free fragment of the Antihydra macro model), `AntihydraSRS.lean` (the SRS itself and its local correctness proof), and `AntihydraSRSSimulation.lean` (the global faithful simulation between arbitrary SRS derivations and the deterministic macro orbit), and `AntihydraSRSForward.lean` (a constructive forward strategy that realises the orbit, giving the exact converse of the faithful-simulation theorem), and `AntihydraSRSObstruction.lean` (a pumping obstruction showing that unbounded future dips of the counter rule out any regular non-halting invariant over unary-counter encodings). This section records the precise definitions and the main lemmas with proofs in mathematical prose.

3.1 String rewriting systems

Let α be a type of symbols. Following `Basic.lean`, a *string rewriting system* (SRS) over α is a binary relation R on $\mathrm{List}\,\alpha$; we write $R\,\ell\,r$ or $\ell \to r \in R$ when (ℓ, r) is a rule. The *rewrite relation* $\to_R$ is defined by

$$u \to_R v \iff \exists\, s, t, \ell, r\, \big(\ell \to r \in R \,\wedge\, u = s\ell t \,\wedge\, v = srt\big). \tag{11}$$

In words: a rule may be applied inside any surrounding context. The following two closure properties are immediate from the definition.

Lemma 3.1 (Rules are rewrite steps)**.** If $\ell \to r \in R$ then $\ell \to_R r$.

Lemma 3.2 (Context closure)**.** If $u \to_R v$ then $sut \to_R svt$ for all strings s, t.

For systems R, S we write $R \cup S$ for the union of their rule sets.

3.2 Mixed-base representations

A *mixed digit* is a pair $(n)_b$ with digit $n \in \mathbb{N}$ and base $b \in \mathbb{N}$; a string of such digits is a mixed-base representation. In the function view of `MixedBaseRepresentation.lean` the symbol $(n)_b$ denotes the affine map

$$\beta_b^n(x) = b \cdot x + n,$$

and a string $N = (n_1)_{b_1} \cdots (n_k)_{b_k}$ denotes the composite

$$\Gamma_N(x) = (\beta_{b_k}^{n_k} \circ \cdots \circ \beta_{b_1}^{n_1})(x), \tag{12}$$

where the leftmost symbol is applied innermost. The *value* of the string is the natural number

$$\mathrm{Val}(N) = \sum_{i=1}^{k} n_i \prod_{j=i+1}^{k} b_j, \tag{13}$$

with the convention that the empty product is 1.

Lemma 3.3 (`gamma_eq`)**.** For every mixed-base string N and every $x \in \mathbb{N}$,

$$\Gamma_N(x) = \big(\prod_{j=1}^{k} b_j\big) \cdot x + \mathrm{Val}(N).$$

Proof. By induction on the length of N. The empty string gives $\Gamma_\emptyset(x) = x = 1 \cdot x + 0$. For $N = (n)_b :: N'$, the induction hypothesis gives $\Gamma_{N'}(y) = (\prod_{j=2}^{k} b_j) \cdot y + \mathrm{Val}(N')$. Taking $y = \beta_b^n(x) = bx + n$ and using the definition of Val yields the claim after collecting the x-term and the constant term. □

Lemma 3.4 (`gamma_headBase_zero`)**.** If the leading base is $b_1 = 0$, then $\Gamma_N(x) = \mathrm{Val}(N)$ for every x.

Proof. In (12) the innermost map is $\beta_0^{n_1}(x) = n_1$, so the whole composite is constant; by the previous lemma the constant is $\mathrm{Val}(N)$. □

The fundamental local operation is *base swapping*: two adjacent symbols can have their bases exchanged without changing the composite, at the cost of updating the digits by division with remainder.

Lemma 3.5 (`baseSwap`)**.** Let $b, c, n, m \in \mathbb{N}$ with $n < b$ and $m < c$. Define

$$n' = \frac{bm + n}{c}, \qquad m' = (bm + n) \bmod c.$$

Then $n' < b$, $m' < c$, and for every x,

$$\beta_b^n\big(\beta_c^m(x)\big) = \beta_c^{m'}\big(\beta_b^{n'}(x)\big).$$

Proof. Both sides equal $bcx + bm + n$. On the right we use the division identity $bm + n = cn' + m'$. The bound $m' < c$ follows from the definition of the remainder. For $n' < b$, note that $m < c$ implies $b(m + 1) \le bc$, so $bm + n < bm + b \le bc$; hence $(bm + n)/c < b$. □

3.3 The Antihydra alphabet and rules

The Antihydra SRS uses the eight-symbol alphabet

$$\Sigma_A = \{f, t, 0, 1, 2, <, >, c\},$$

where in the Lean code the ternary digits are named $d0, d1, d2$ and the boundary markers are lhd, rhd. The symbol maps are

symbol s	$\mathrm{symFun}_s(x)$
f	$2x$
t	$2x + 1$
0	$3x$
1	$3x + 1$
2	$3x + 2$
$<$	1
$>$	x
c	x

(14)

Thus $<$ contributes the initial value 1; $>$ returns the current value; and the counter symbol c is the identity. For a word $w \in \mathrm{List}\,\Sigma_A$ and a starting value x, the composite map is the left fold

$$\mathrm{compFun}_w(x) = w.\,\mathrm{foldl}\big((acc, s) \mapsto \mathrm{symFun}_s(acc)\big)(x).$$

Lemma 3.6 (`compFun_cons` and `compFun_append`)**.** For all symbols s, words u, v, and values x,

$$\mathrm{compFun}_{s::w}(x) = \mathrm{compFun}_w\big(\mathrm{symFun}_s(x)\big)$$

and

$$\mathrm{compFun}_{u^\frown v}(x) = \mathrm{compFun}_v\big(\mathrm{compFun}_u(x)\big).$$

Proof. Both are immediate from the definition of a left fold and the associativity of list append. □

A *configuration* is a string of the form

$$\operatorname{config} wb =< \ w \ > \ c^b, \tag{15}$$

where $w \in \{f, t, 0, 1, 2\}^*$ is the value word and $b \in \mathbb{N}$ is the counter. The value of the configuration is independent of the counter.

Lemma 3.7 (`cfgVal`)**.** For every value word w, counter b, and value $x \in \mathbb{N}$,

$$\operatorname{Val}(\operatorname{config} wb) = \operatorname{compFun}_{<w>}(x).$$

Proof. Since $\operatorname{symFun}_c(x) = x$, the block c^b is transparent: `compFun_replicate_c` proves $\operatorname{compFun}_{c^b}(x) = x$ by induction on b. Lemma 3.6 therefore gives

$$\operatorname{compFun}_{<w>c^b}(x) = \operatorname{compFun}_{c^b}\big(\operatorname{compFun}_{<w>}(x)\big) = \operatorname{compFun}_{<w>}(x). \qquad \square$$

The counter is measured by the number of c symbols:

$$\operatorname{countC}(w) = \text{the number of occurrences of } c \text{ in } w.$$

It is additive over concatenation: $\operatorname{countC}(uv) = \operatorname{countC}(u) + \operatorname{countC}(v)$.

The rewriting system is the union of three groups of rules.

Dynamic rules.

$$\mathcal{D}: \quad f >\!\!\rightarrow 0 > cc, \qquad t > c \rightarrow 1 > . \tag{16}$$

Transport rules.

$$\mathcal{A}: \quad \begin{array}{lll} f0 \rightarrow 0f, & f1 \rightarrow 0t, & f2 \rightarrow 1f, \\ t0 \rightarrow 1t, & t1 \rightarrow 2f, & t2 \rightarrow 2t. \end{array} \tag{17}$$

Boundary rules.

$$\mathcal{B}: \quad < 0 \rightarrow\!\!< t, \qquad < 1 \rightarrow\!\!< ff, \qquad < 2 \rightarrow\!\!< ft. \tag{18}$$

The auxiliary subsystem is $\mathcal{X} = \mathcal{A} \cup \mathcal{B}$, and the full Antihydra SRS is

$$\mathcal{R}_A = \mathcal{D} \cup \mathcal{X}.$$

3.4 The macro model

To state what the SRS represents, we first recall the integer recurrence from Section 1.2 in the mixed-base value coordinate. Define the (partial) *macro step* $\operatorname{Step} : \mathbb{N} \times \mathbb{N} \rightharpoonup \mathbb{N} \times \mathbb{N}$ by

$$\operatorname{Step}(a, b) = \begin{cases} \big(\lfloor 3a/2 \rfloor, \ b+2\big), & a \text{ even}, \\ \text{undefined}, & a \text{ odd and } b = 0, \\ \big(\lfloor 3a/2 \rfloor, \ b-1\big), & a \text{ odd and } b > 0. \end{cases} \tag{19}$$

Equivalently, writing $a = 2x$ or $a = 2x+1$,

$$\operatorname{Step}(2x, b) = (3x, b+2), \qquad \operatorname{Step}(2x+1, 0) = \text{undefined}, \qquad \operatorname{Step}(2x+1, b+1) = (3x+1, b). \tag{20}$$

In the file `AntihydraSRSaxiom.lean` the same machine is presented in a different, but equivalent, coordinate: the state is a pair (a, b) where a is the length m of the left block 1^m in the macro configuration $P(b, m, n, p)$ of (1), and the total map is

$$A(a,b) = \big(3 \cdot (a/2) + 2,\ b + 2\big) \quad \text{if } a \text{ even}, \qquad A(a,b) = \big(3 \cdot (a/2) + 3,\ b - 1\big) \quad \text{if } a \text{ odd}.$$

The partial step nextMathState returns none exactly when a is odd and $b = 0$. The reason for the $+2/+3$ offsets is that this model tracks the length m directly, whereas the mixed-base model tracks the value $\mathrm{Val}(< w >)$. Both models agree on the halting condition, and the bridge is supplied by the BusyLean proof in `AntihydraMachine.lean` (see Section 3.8). Our SRS correctness theorem is stated in the cleaner mixed-base coordinate.

3.5 Correctness: the SRS represents the macro step

The correctness proof has three ingredients: the auxiliary rules preserve value and counter, the dynamic rules implement the two branches of Step, and the halting condition matches.

Lemma 3.8 (`auxRules_preserve_value`)**.** For every rule $\ell \to r \in \mathcal{X}$ and every $x \in \mathbb{N}$, $\mathrm{compFun}_\ell(x) = \mathrm{compFun}_r(x)$.

Proof. There are nine rules. Recall that compFun is a left fold, so a string $s_1 s_2 \dots s_k$ means: start with x, apply s_1, then s_2, and so on, finishing with s_k; the final $>$ returns the current accumulator.

Transport rules. The six transport rules are the special cases of Lemma 3.5 for bases 2 and 3. Writing the left-fold computation in each case gives

$$\begin{aligned}
f0(x) &= 3 \cdot (2x) = 6x, & 0f(x) &= 2 \cdot (3x) = 6x,\\
f1(x) &= 3 \cdot (2x) + 1 = 6x + 1, & 0t(x) &= 2 \cdot (3x) + 1 = 6x + 1,\\
f2(x) &= 3 \cdot (2x) + 2 = 6x + 2, & 1f(x) &= 2 \cdot (3x + 1) = 6x + 2,\\
t0(x) &= 3 \cdot (2x + 1) = 6x + 3, & 1t(x) &= 2 \cdot (3x + 1) + 1 = 6x + 3,\\
t1(x) &= 3 \cdot (2x + 1) + 1 = 6x + 4, & 2f(x) &= 2 \cdot (3x + 2) = 6x + 4,\\
t2(x) &= 3 \cdot (2x + 1) + 2 = 6x + 5, & 2t(x) &= 2 \cdot (3x + 2) + 1 = 6x + 5.
\end{aligned}$$

In every pair the two composites agree.

Boundary rules. Here the leftmost symbol $<$ replaces the input by 1, after which the remaining digits are applied in order and $>$ returns the result. Thus

$$\begin{aligned}
\mathrm{compFun}_{[<,0]}(x) &= 0(1) = 3, & \mathrm{compFun}_{[<,t]}(x) &= t(1) = 2 \cdot 1 + 1 = 3,\\
\mathrm{compFun}_{[<,1]}(x) &= 1(1) = 4, & \mathrm{compFun}_{[<,f,f]}(x) &= f(f(1)) = 2 \cdot 2 = 4,\\
\mathrm{compFun}_{[<,2]}(x) &= 2(1) = 5, & \mathrm{compFun}_{[<,f,t]}(x) &= t(f(1)) = 2 \cdot 2 + 1 = 5.
\end{aligned}$$

All three boundary rules therefore preserve value. □

Lemma 3.9 (`auxRules_preserve_count`)**.** Every rule $\ell \to r \in \mathcal{X}$ satisfies $\mathrm{countC}(\ell) = \mathrm{countC}(r)$.

Proof. None of the nine auxiliary rules mentions the counter symbol c. □

Because rewriting is applied inside a context, preservation extends from rules to arbitrary rewrite steps.

Theorem 3.10 (`auxRules_rewriteStep_preserve_value` and `_preserve_count`)**.** If $u \to_{\mathcal{X}} v$, then $\mathrm{compFun}_u(x) = \mathrm{compFun}_v(x)$ for all x and $\mathrm{countC}(u) = \mathrm{countC}(v)$.

Proof. Write $u = s\ell t$ and $v = srt$ with $\ell \to r \in \mathcal{X}$. Put $y = \mathrm{compFun}_s(x)$. By Lemma 3.6,

$$\mathrm{compFun}_u(x) = \mathrm{compFun}_t\big(\mathrm{compFun}_\ell(y)\big) \text{ and } \mathrm{compFun}_v(x) = \mathrm{compFun}_t\big(\mathrm{compFun}_r(y)\big),$$

which are equal by Lemma 3.8. The counter equality follows from $\mathrm{countC}(uv) = \mathrm{countC}(u) + \mathrm{countC}(v)$ and Lemma 3.9. □

Lemma 3.11 (`dynEven`)**.** For every prefix word pre and every $x \in \mathbb{N}$,

$$\begin{aligned}\mathrm{compFun}_{\mathrm{pre}^\frown f>}(x) &= 2 \cdot \mathrm{compFun}_{\mathrm{pre}}(x),\\ \mathrm{compFun}_{\mathrm{pre}^\frown 0>cc}(x) &= 3 \cdot \mathrm{compFun}_{\mathrm{pre}}(x),\\ \mathrm{countC}(\mathrm{pre}^\frown 0 > cc) &= \mathrm{countC}(\mathrm{pre}^\frown f >) + 2.\end{aligned}$$

Proof. Direct computation. We have $\mathrm{compFun}_{\mathrm{pre}^\frown f>}(x) => \big(\mathrm{symFun}_f(\mathrm{compFun}_{\mathrm{pre}}(x))\big) = 2 \cdot \mathrm{compFun}_{\mathrm{pre}}(x)$, and similarly $\mathrm{compFun}_{\mathrm{pre}^\frown 0>cc}(x) => \big(\mathrm{symFun}_0(\mathrm{compFun}_{\mathrm{pre}}(x))\big) = 3 \cdot \mathrm{compFun}_{\mathrm{pre}}(x)$ because the trailing cc is transparent. The counter changes by exactly the two new c symbols. □

Lemma 3.12 (`dynOdd`)**.** For every prefix word pre and every $x \in \mathbb{N}$,

$$\begin{aligned}\mathrm{compFun}_{\mathrm{pre}^\frown t>c}(x) &= 2 \cdot \mathrm{compFun}_{\mathrm{pre}}(x) + 1,\\ \mathrm{compFun}_{\mathrm{pre}^\frown 1>}(x) &= 3 \cdot \mathrm{compFun}_{\mathrm{pre}}(x) + 1,\\ \mathrm{countC}(\mathrm{pre}^\frown t > c) &= \mathrm{countC}(\mathrm{pre}^\frown 1 >) + 1.\end{aligned}$$

Proof. Same direct computation, using $\mathrm{symFun}_t(y) = 2y + 1$ and $\mathrm{symFun}_1(y) = 3y + 1$. □

Together, Lemmas 3.11 and 3.12 say that a dynamic firing on a configuration changes the value word from a binary least significant digit f or t to a ternary least significant digit 0 or 1, while applying exactly the map $a \mapsto \lfloor 3a/2 \rfloor$ and updating the counter by $+2$ or -1.

Theorem 3.13 (`antihydraSRS_represents_macro`)**.** The Antihydra SRS represents the macro step Step in the following sense.

1. Every auxiliary rewrite preserves value and counter: if $u \to_{\mathcal{X}} v$ then $\mathrm{compFun}_u(x) = \mathrm{compFun}_v(x)$ for all x and $\mathrm{countC}(u) = \mathrm{countC}(v)$.
2. The even dynamic rule realises $(2x, b) \mapsto (3x, b + 2)$: for every prefix pre,
$$\begin{aligned}\mathrm{compFun}_{\mathrm{pre}^\frown f>}(x) &= 2 \cdot \mathrm{compFun}_{\mathrm{pre}}(x),\\ \mathrm{compFun}_{\mathrm{pre}^\frown 0>cc}(x) &= 3 \cdot \mathrm{compFun}_{\mathrm{pre}}(x),\end{aligned}$$
and the counter grows by 2.
3. The odd dynamic rule realises $(2x + 1, b + 1) \mapsto (3x + 1, b)$: for every prefix pre,
$$\begin{aligned}\mathrm{compFun}_{\mathrm{pre}^\frown t>c}(x) &= 2 \cdot \mathrm{compFun}_{\mathrm{pre}}(x) + 1,\\ \mathrm{compFun}_{\mathrm{pre}^\frown 1>}(x) &= 3 \cdot \mathrm{compFun}_{\mathrm{pre}}(x) + 1,\end{aligned}$$
and the counter drops by 1.

Proof. Part (1) is Theorem 3.10; parts (2) and (3) are Lemmas 3.11 and 3.12. □

3.6 Faithful simulation

Theorem 3.13 shows that each rule of the SRS does the right thing *when it fires*. It does not yet say that an arbitrary SRS derivation must follow the deterministic macro orbit: a derivation might, in principle, fire dynamic rules at unexpected positions or in an order inconsistent with the value. The file `AntihydraSRSSimulation.lean` closes this gap by proving that *every* derivation from a configuration is faithful to the orbit.

A *digit string* is a word $w \in \{f, t, 0, 1, 2\}^*$ containing no markers or counter symbol. A *configuration* is therefore $\mathrm{config}\, wb$ with w a digit string. The first structural fact is that a dynamic redex is forced to the unique boundary marker $>$.

Lemma 3.14 (`dynStep_config_dest`)**.** Let w be a digit string and $b \in \mathbb{N}$. If $\mathrm{config}\, wb \to_{\mathcal{D}} v$, then either

- $w = w'^\frown f$ and $v = \mathrm{config}\,(w'^\frown 0)(b+2)$, or
- $w = w'^\frown t$, $b = b' + 1$, and $v = \mathrm{config}\,(w'^\frown 1)b'$.

Proof sketch. A dynamic rule contains the marker $>$ in its left-hand side, while a configuration contains exactly one $>$ (because w is a digit string). By counting occurrences of $>$ and using injectivity of the split around that symbol, the redex must be positioned so that the rule's $>$ coincides with the configuration's unique $>$. Hence the rule sees the last symbol of w; if it is f the even rule fires, if it is t the odd rule fires and consumes one counter symbol. □

Consequently the rule that fires is determined by the parity of the value $\mathrm{Val}(\mathrm{config}\, wb)$, not by the rewriting strategy.

Lemma 3.15 (`dynStep_config`)**.** Let w be a digit string and $b \in \mathbb{N}$. If $\mathrm{config}\, wb \to_{\mathcal{D}} v$, then $v = \mathrm{config}\, w'b'$ for some digit string w' and counter b' with

$$\mathrm{Val}(\mathrm{config}\, w'b') = \left\lfloor \frac{3\ \mathrm{Val}(\mathrm{config}\, wb)}{2} \right\rfloor$$

and

$$\begin{cases} b' = b + 2, & \mathrm{Val}(\mathrm{config}\, wb) \text{ even}, \\ b = b' + 1, & \mathrm{Val}(\mathrm{config}\, wb) \text{ odd}. \end{cases}$$

Proof. By Lemma 3.14 the rewrite is either the even rule $w = w'^\frown f \mapsto w'^\frown 0$ with $b' = b+2$, or the odd rule $w = w'^\frown t \mapsto w'^\frown 1$ with $b = b' + 1$. The value equations are exactly Lemmas 3.11 and 3.12. □

Auxiliary steps are even better behaved: they never change the shape of a configuration.

Lemma 3.16 (`auxStep_config_dest`)**.** Let w be a digit string and $b \in \mathbb{N}$. If $\mathrm{config}\, wb \to_{\mathcal{X}} v$, then $v = \mathrm{config}\, w'b$ for some digit string w'.

Proof sketch. Transport rules have left-hand sides consisting of two digit symbols with no marker, so a redex inside $\mathrm{config}\, wb$ must lie entirely inside the digit block w; boundary rules have left-hand side $< d$ with d a digit, so they rewrite the first digit of w. Neither kind mentions c, so the counter b is unchanged. □

Putting the two kinds of step together gives the main simulation theorem. We write $\mathrm{valOrbit}\, a_0 n$ for the n-th iterate of $a \mapsto \lfloor 3a/2 \rfloor$ starting from a_0, and $\mathrm{counterZ}\, b_0 a_0 n$ for the counter after n macro steps when the initial counter is b_0.

Theorem 3.17 (`antihydraSRS_simulates`)**.** Let w_0 be a digit string and $b_0 \in \mathbb{N}$, and let $a_0 = \mathrm{Val}(\mathrm{config}\, w_0 b_0)$. For every string C reachable from $\mathrm{config}\, w_0 b_0$ by the full SRS $\mathcal{R}_A$ there exist a digit string w, a counter b, and a number $k \in \mathbb{N}$ of dynamic steps such that

$$C = \mathrm{config}\, w b, \qquad \mathrm{Val}(\mathrm{config}\, w b) = \mathrm{valOrbit}\, a_0 k, \qquad b = \mathrm{counterZ}\, b_0 a_0 k.$$

In particular, the value and counter after k dynamic steps are independent of the auxiliary rewriting strategy.

Proof. By induction on the reflexive-transitive closure of the rewrite relation. The base case $k = 0$ is trivial. For the inductive step, use Lemma 3.16 for auxiliary steps (they keep k and the counter unchanged and preserve the value by Theorem 3.10) and Lemma 3.15 for a dynamic step (it increments k by one and updates value and counter by exactly one macro step). □

Since the value orbit is deterministic, the number of even and odd steps among the first k dynamic steps is fixed. Theorem 3.17 therefore implies the strategy-independence of the counter in a form that refers directly to SRS derivations.

Corollary 3.18 (`counter_strategy_independent`)**.** Let w_0 be a digit string and $b_0 \in \mathbb{N}$, and let $a_0 = \mathrm{Val}(\mathrm{config}\, w_0 b_0)$. For every configuration C reachable from $\mathrm{config}\, w_0 b_0$ there exist k and j such that

$$C = \mathrm{config}\, w b \quad \text{and} \quad b = b_0 + 2k - 3j,$$

where j is the number of odd steps among the first k macro steps from a_0.

Proof. Theorem 3.17 gives k with $b = \mathrm{counterZ}\, b_0 a_0 k$. Writing e for the number of even steps and j for the number of odd steps among the first k steps, we have $e + j = k$ and $b = b_0 + 2e - j = b_0 + 2k - 3j$. □

Comparison with the soundness layer.

The file `AntihydraSRS.lean` proves local soundness: each rule preserves or correctly updates value and counter. The file `AntihydraSRSSimulation.lean` lifts those local facts to a global statement about arbitrary derivations. This is exactly the step from YAH's Claim 1 (per-rule correctness) to the general simulation lemma of Theorem 3.17.

We can also exhibit the macro step as an explicit one-step rewrite on configurations.

Lemma 3.19 (`config_even_step`)**.** For every value word w and every counter b,

$$\mathrm{config}\,(w^\frown f) b \to_{\mathcal{R}_A} \mathrm{config}\,(w^\frown 0) b + 2.$$

Proof. Apply the rule $f > \!\!\to\, 0 > cc$ in the context $< w$ on the left and c^b on the right. Formally,

$$\mathrm{config}\,(w^\frown f) b = (< w)\,(f >)\, c^b, \qquad \mathrm{config}\,(w^\frown 0) b + 2 = (< w)\,(0 > cc)\, c^b.$$

□

Lemma 3.20 (`config_odd_step`)**.** For every value word w and every counter b,

$$\mathrm{config}\,(w^\frown t) b + 1 \to_{\mathcal{R}_A} \mathrm{config}\,(w^\frown 1) b.$$

Proof. Apply the rule $t > c \to 1 >$ in the context $< w$ on the left and c^b on the right. The trailing c of the rule consumes one counter symbol from the c^{b+1} block, leaving c^b. □

Lemma 3.21 (`dynRules_odd_needs_counter`)**.** The string $t >$ is not the left-hand side of any dynamic rule. Consequently, a configuration of the form $u\,t >$ with no counter symbol is irreducible.

Proof. The two dynamic left-hand sides are $f >$ and $t > c$; neither equals $t >$. □

The preceding lemma matches the macro halting condition.

Theorem 3.22 (`nextMathState_halt_condition`)**.** For $a \geq 2$, the BusyLean-free macro model halts with empty counter, $\mathrm{nextMathState}(a, 0) = \mathrm{none}$, if and only if a is odd.

Proof. By definition $\mathrm{nextMathState}(a, 0)$ returns none exactly on the odd branch of the conditional, which is taken precisely when a is odd. (The hypothesis $a \geq 2$ guarantees that the outer guard $a/2 \geq 1$ holds.) □

Thus an odd value with an empty counter is a halting configuration both for the SRS (Lemma 3.21) and for the macro model (Theorem 3.22).

3.7 Forward (constructive) simulation

Theorem 3.17 is the *backward* half of a simulation theorem: every SRS derivation stays inside the deterministic orbit. It is compatible, in principle, with the SRS making no progress at all. The file `AntihydraSRSForward.lean` supplies the missing *forward* half: an explicit, essentially deterministic rewriting strategy that walks the orbit step by step. Together the two halves give an exact characterisation of the reachable configurations.

3.7.1 Binary digit strings

For the forward construction it is convenient to restrict attention to value words that contain only binary digits.

Definition 3.23 (`IsBinary`)**.** A list $w \in \mathrm{List}\,\Sigma_A$ is *binary* if every symbol of w is f or t.

Lemma 3.24 (`IsBinary.isDigits`)**.** Every binary string is a digit string.

Proof. A binary symbol is either f or t, both of which are digits. □

Lemma 3.25 (`IsBinary.snoc`)**.** If w is binary and s is f or t, then $w^\frown[s]$ is binary.

Proof. Immediate from the definition. □

Lemma 3.26 (`IsBinary.exists_snoc`)**.** Every nonempty binary string w splits uniquely as $w = p^\frown[s]$ where $s \in \{f, t\}$ and p is binary.

Proof. Take s to be the last symbol and p the prefix; the conditions are satisfied by construction, and uniqueness follows from the fact that the last symbol of a list is unique. □

On a binary string w the value function has the simple form

$$\mathrm{Val}(< w >) = \mathrm{compFun}_w(1),$$

which is exactly the natural number whose binary expansion is $1^\frown w$ (with $f = 0$ and $t = 1$). Hence the parity of the value is read off the last symbol: f gives an even value and t gives an odd value.

3.7.2 Lifting auxiliary derivations

The carry sweep below is built purely from the auxiliary rules $\mathcal{X} = \mathcal{A} \cup \mathcal{B}$. To make it a derivation of the full SRS we need a trivial monotonicity lemma.

Lemma 3.27 (`auxStep_to_antihydra`)**.** Every single $\mathcal{X}$-rewrite step is an $\mathcal{R}_A$-rewrite step.

Proof. The full rule set is $\mathcal{R}_A = \mathcal{D} \cup \mathcal{X}$, so every rule of $\mathcal{X}$ is also a rule of $\mathcal{R}_A$. □

Theorem 3.28 (`auxDeriv_to_antihydra`)**.** If $u \to_{\mathcal{X}}^{*} v$, then $u \to_{\mathcal{R}_A}^{*} v$.

Proof. Reflexive-transitive closure is monotone with respect to inclusion of the underlying relation. Lemma 3.27 gives the inclusion of single-step relations. □

3.7.3 The carry sweep

A dynamic firing replaces the rightmost binary digit of w by a single ternary digit ($f \mapsto 0$, $t \mapsto 1$). Before the next dynamic step can fire, that ternary digit must be carried leftward through the binary block until it reaches the left boundary $<$, where it is resolved back into binary digits. The transport rules $\mathcal{A}$ perform one leftward swap at a time.

Lemma 3.29 (`auxA_swap`)**.** Let $s \in \{f, t\}$ be a binary symbol and $d_k \in \{0, 1, 2\}$ a ternary symbol. Then there exist a ternary symbol d_j and a binary symbol s_j such that $s\, d_k \to d_j\, s_j$ is a rule of $\mathcal{A}$ and, for all $x \in \mathbb{N}$,

$$\mathrm{symFun}_{d_k}\big(\mathrm{symFun}_{s}(x)\big) = \mathrm{symFun}_{s_j}\big(\mathrm{symFun}_{d_j}(x)\big).$$

Proof. There are six cases, corresponding to the six transport rules in (17). Each is a direct computation. For instance, for $t1 \to 2f$ we have $\mathrm{symFun}_1(\mathrm{symFun}_t(x)) = 3(2x+1)+1 = 6x+4 = 2(3x+2) = \mathrm{symFun}_f(\mathrm{symFun}_2(x))$. The other five cases are analogous. □

Theorem 3.30 (`carry_sweep`)**.** Let p be a binary string and $d_k \in \{0, 1, 2\}$. For every suffix *rest* there exists a binary string q such that

$$\mathrm{Val}(< q >) = \mathrm{symFun}_{d_k}\big(\mathrm{Val}(< p >)\big)$$

and

$$< p\, d_k\, \mathit{rest} \to_{\mathcal{X}}^{*} < q\, \mathit{rest}.$$

Proof. By reverse induction on p. If $p = \varepsilon$, the string is $< d_k\, \mathit{rest}$ and a single boundary rule resolves it: $< 0 \to < t$, $< 1 \to < ff$, or $< 2 \to < ft$. The resulting q is $[t]$, $[f, f]$, or $[f, t]$, and the value equation is immediate.

If $p = p' ^\frown [s]$ with $s \in \{f, t\}$, Lemma 3.29 gives d_j and s_j with $s\, d_k \to d_j\, s_j \in \mathcal{A}$ and equal composite functions. One transport step yields

$$< p'\, s\, d_k\, \mathit{rest} \to_{\mathcal{A}} < p'\, d_j\, s_j\, \mathit{rest}.$$

The induction hypothesis applied to p', d_j, and $\mathit{rest}' = s_j\, \mathit{rest}$ gives a binary q' with $\mathrm{Val}(< q' >) = \mathrm{symFun}_{d_j}(\mathrm{Val}(< p' >))$ and $< p'\, d_j\, s_j\, \mathit{rest} \to_{\mathcal{X}}^{*} < q'\, s_j\, \mathit{rest}$. Set $q = q' ^\frown [s_j]$. Then q is binary by Lemma 3.26, and

$$\begin{aligned}
\mathrm{Val}(< q >) &= 2\ \mathrm{Val}(< q' >) + \delta \\
&= \mathrm{symFun}_{s_j}\big(\mathrm{symFun}_{d_j}(\mathrm{Val}(< p' >))\big) \\
&= \mathrm{symFun}_{d_k}\big(\mathrm{symFun}_{s}(\mathrm{Val}(< p' >))\big) \\
&= \mathrm{symFun}_{d_k}\big(\mathrm{Val}(< p >)\big).
\end{aligned}$$

where $\delta = 0$ if $s_j = f$ and $\delta = 1$ if $s_j = t$. Concatenating the single transport step with the induction hypothesis gives the required derivation. □

3.7.4 The macro orbit as an option iteration

To state the forward theorem compactly, the Lean file defines an iteration of the partial macro step.

Definition 3.31 (`macroIter`)**.** Define $\mathrm{macroIter} : \mathbb{N} \to (\mathbb{N} \times \mathbb{N}) \to \mathrm{Option}(\mathbb{N} \times \mathbb{N})$ by

$$\mathrm{macroIter}(0, s) = \mathrm{some}(s), \qquad \mathrm{macroIter}(n+1, s) = \mathrm{macroIter}(n, s) \gg\!= \mathrm{Step},$$

where $u \gg\!= f$ is none if $u = \mathrm{none}$ and $f(a, b)$ if $u = \mathrm{some}(a, b)$.

Thus $\mathrm{macroIter}(n, (a_0, b_0)) = \mathrm{some}(a_n, b_n)$ means precisely that the macro model has not halted within the first n steps and that (a_n, b_n) is the state after n steps.

3.7.5 The forward simulation theorem

We can now prove that the SRS does not merely stay inside the orbit, but actually walks it.

Theorem 3.32 (`antihydraSRS_realizes_orbit`)**.** Let w_0 be a binary string and $b_0 \in \mathbb{N}$, and assume $a_0 = \mathrm{Val}(< w_0 >) \geq 2$. For every $n, a_n, b_n \in \mathbb{N}$ with

$$\mathrm{macroIter}(n, (a_0, b_0)) = \mathrm{some}(a_n, b_n),$$

there exists a binary string w_n such that

$$\mathrm{Val}(< w_n >) = a_n, \qquad \mathrm{Val}(< w_n >) \geq 2,$$

and

$$\mathrm{config}\, w_0 b_0 \to^*_{\mathcal{R}_A} \mathrm{config}\, w_n b_n.$$

Proof. Induction on n. The base case $n = 0$ is trivial: take $w_0 = w_n$.

For the inductive step, suppose $\mathrm{macroIter}(n+1, (a_0, b_0)) = \mathrm{some}(a_{n+1}, b_{n+1})$. Unfolding the iteration gives (a_n, b_n) with $\mathrm{macroIter}(n, (a_0, b_0)) = \mathrm{some}(a_n, b_n)$ and $\mathrm{Step}(a_n, b_n) = \mathrm{some}(a_{n+1}, b_{n+1})$. By the induction hypothesis there is a binary w_n with $\mathrm{Val}(< w_n >) = a_n \geq 2$ and $\mathrm{config}\, w_0 b_0 \to^*_{\mathcal{R}_A} \mathrm{config}\, w_n b_n$.

Because $a_n \geq 2$, the word w_n is nonempty, so by Lemma 3.26 it splits as $w_n = p^\frown[s]$ with $s \in \{f, t\}$. Consider the two parity cases.

Even case ($s = f$). Here $\mathrm{Val}(< w_n >) = 2\ \mathrm{Val}(< p >)$, so $a_n = 2\ \mathrm{Val}(< p >)$ and

$$\mathrm{Step}(a_n, b_n) = (3\ \mathrm{Val}(< p >),\ b_n + 2).$$

Lemma 3.19 gives the single dynamic step $\mathrm{config}\,(p^\frown[f])b_n \to_{\mathcal{R}_A} \mathrm{config}\,(p^\frown[0])b_n + 2$. Theorem 3.30, with $d_k = 0$ and $rest => c^{b_n+2}$, yields a binary q with $\mathrm{Val}(< q >) = 3\ \mathrm{Val}(< p >)$ and $< p\,0 > c^{b_n+2} \to^*_{\mathcal{X}} < q > c^{b_n+2}$. By Theorem 3.28 this is also a derivation of $\mathcal{R}_A$, so $\mathrm{config}\,(p^\frown[0])b_n + 2 \to^*_{\mathcal{R}_A} \mathrm{config}\, q b_n + 2$. Set $w_{n+1} = q$; then $(a_{n+1}, b_{n+1}) = (\mathrm{Val}(< q >), b_n + 2)$ as required.

Odd case ($s = t$). Here $\mathrm{Val}(< w_n >) = 2\ \mathrm{Val}(< p >) + 1$, so $a_n = 2\ \mathrm{Val}(< p >) + 1$. Since $\mathrm{Step}(a_n, b_n)$ is defined, the counter is positive; write $b_n = b' + 1$. Then

$$\mathrm{Step}(a_n, b_n) = (3\ \mathrm{Val}(< p >) + 1,\ b').$$

Lemma 3.20 gives $\mathrm{config}\,(p^\frown[t])b' + 1 \to_{\mathcal{R}_A} \mathrm{config}\,(p^\frown[1])b'$, and Theorem 3.30 with $d_k = 1$ carries the digit 1 to a binary q with $\mathrm{Val}(< q >) = 3\ \mathrm{Val}(< p >) + 1$. Again set $w_{n+1} = q$.

In both cases we have extended the derivation from $\mathrm{config}\, w_0 b_0$ to $\mathrm{config}\, w_{n+1} b_{n+1}$, completing the induction. □

The value produced by macroIter is exactly the deterministic value orbit.

Lemma 3.33 (`macroIter_fst_eq_valOrbit`)**.** For all a_0, b_0, n, a_n, b_n, if

$$\mathrm{macroIter}(n, (a_0, b_0)) = \mathrm{some}(a_n, b_n),$$

then $a_n = \mathrm{valOrbit}\, a_0 n$.

Proof. By induction on n, using the fact that Step applies the map $a \mapsto \lfloor 3a/2 \rfloor$ on both the even and the odd branch (Lemmas 3.11 and 3.12). □

Repackaging Theorem 3.32 against the value orbit gives the following direct converse of Theorem 3.17.

Theorem 3.34 (`antihydraSRS_realizes_valOrbit`)**.** Let w_0 be binary with $a_0 = \mathrm{Val}(< w_0 >) \geq 2$, and let $\mathrm{macroIter}(n, (a_0, b_0)) = \mathrm{some}(a_n, b_n)$. Then there exists a binary w_n such that

$$\mathrm{Val}(< w_n >) = \mathrm{valOrbit}\, a_0 n \qquad \text{and} \qquad \mathrm{config}\, w_0 b_0 \to^*_{\mathcal{R}_A} \mathrm{config}\, w_n b_n.$$

Proof. Combine Theorem 3.32 with Lemma 3.33. □

Finally, the backward and forward theorems together give an exact characterisation of reachability.

Theorem 3.35 (`antihydraSRS_orbit_exact`)**.** Let w_0 be binary with $a_0 = \mathrm{Val}(< w_0 >) \geq 2$ and let $\mathrm{macroIter}(n, (a_0, b_0)) = \mathrm{some}(a_n, b_n)$. Then:

1. There exists a binary w_n with $\mathrm{Val}(< w_n >) = \mathrm{valOrbit}\, a_0 n$ and $\mathrm{config}\, w_0 b_0 \to^*_{\mathcal{R}_A} \mathrm{config}\, w_n b_n$.
2. Conversely, for every configuration C reachable from $\mathrm{config}\, w_0 b_0$ there exist k, w, and b such that $C = \mathrm{config}\, w b$ and $\mathrm{Val}(< w >) = \mathrm{valOrbit}\, a_0 k$.

In other words, the configurations reachable from $\mathrm{config}\, w_0 b_0$ are exactly the orbit configurations.

Proof. Part (1) is Theorem 3.34; part (2) is Theorem 3.17. □

Corollary 3.36 (`antihydraSRS_realizes_orbit_from_eight`)**.** For every b_0, n, a_n, b_n with $\mathrm{macroIter}(n, (8, b_0)) = \mathrm{some}(a_n, b_n)$, there exists a binary w_n such that $\mathrm{Val}(< w_n >) = \mathrm{valOrbit}\, 8n$ and $\mathrm{config}\, [f, f, f] b_0 \to^*_{\mathcal{R}_A} \mathrm{config}\, w_n b_n$.

Proof. The word $[f, f, f]$ is binary and $\mathrm{Val}(< fff >) = 8$, so Theorem 3.34 applies. □

Comparison with the soundness and backward layers

The chain of files is now complete. `AntihydraSRS.lean` proves local soundness (each rule preserves or correctly updates value and counter); `AntihydraSRSSimulation.lean` lifts this to the global backward statement that arbitrary derivations are faithful to the orbit; and `AntihydraSRSForward.lean` supplies the constructive forward strategy that realises the orbit. Theorem 3.35 is the exact analogue, for Antihydra, of YAH's Theorem 3.17 in full: Claims 1 and 2 give the backward simulation, and Claim 3 gives the forward realisation.

3.8 Bridge to the Turing machine

The file `AntihydraMachine.lean` contains the BusyLean proof that the macro model simulates the actual Antihydra Turing machine. The main ingredients are the six macro theorems of Section 1.1, which are lifted from finite padded configurations $P(b, m, n, p)$ to stream configurations $SP(b, m, n)$ and assembled into the simulation theorem `stm_simulates_math`. This theorem says that for every $a \geq 2$ and every b, there is a positive number of TM steps k such that the machine from $SP(b, a, 0)$ reaches $SP(b', a', 0)$ if $\mathrm{nextMathState}(a, b) = \mathrm{some}(a', b')$, and halts if $\mathrm{nextMathState}(a, b) = \mathrm{none}$.

The halting predicate mathHalts is defined inductively from nextMathState in the usual way, and the crucial bridge (`mathHalts_iff_Aiter_8_2` in `AntihydraSRSaxiom.lean`) states that

$$\mathrm{mathHalts}(8, 2) \iff \exists i \, \big(A^i(8,2).1 \text{ is odd} \ \wedge \ A^i(8,2).1/2 \geq 1 \ \wedge \ A^i(8,2).2 = 0\big), \quad (21)$$

where A^i is the i-th iterate of the total macro map A. In `AntihydraMachine.lean` this equivalence is proved directly from the definitions of nextMathState and mathHalts, and then combined with `stm_simulates_math` to give the main TM-level result `antihydra_halts_iff`: the Antihydra TM halts if and only if the right-hand side of (21) holds.

Composing this with Theorem 3.13 and Theorem 3.22, we obtain the intended interpretation of the SRS:

Corollary 3.37. The Antihydra SRS is non-terminating from the initial configuration $< fff >$ if and only if the Antihydra Turing machine does not halt.

Proof sketch. By Lemma 3.38 below, $< fff >$ represents $(a, b) = (8, 0)$. The mixed-base macro step Step and the m-coordinate partial step nextMathState have the same halting condition (Theorem 3.22), and the total map A is the totalisation of nextMathState. The BusyLean proof identifies non-halting of the TM with the statement that no iterate of A from the appropriate starting point reaches an odd value with empty counter. Hence a non-terminating SRS derivation corresponds exactly to an infinite Antihydra orbit, and a halting SRS configuration corresponds exactly to a halting TM configuration. □

Relation to the general YAH simulation lemma. The proof template used above is the Antihydra analogue of Theorem 3.17 of Yolcu–Aaronson–Heule [1, pp. 20–21]: one introduces a mixed-base SRS whose auxiliary rules preserve the represented integer value while the dynamic rules implement the iterated map, then proves that the SRS is non-terminating exactly when the original system does not halt. In the Antihydra case the dynamic rules realise the lower 3/2-map with a unary counter, whereas YAH's original system realises the Collatz map T. The same strategy-independence, value-preservation, and halting-characterization lemmas apply whenever the macro dynamics can be written as a piecewise-affine digit update on a mixed-base representation. The broader research program, recorded in Report B3 (§8) and in `3-forward-strategy.html` (§5) [10, 11], is to extract this pattern into a single general simulation lemma for the full YAH rewriting framework, prove it once in Lean, and use it as a verified front-end for all certificate-producing termination proofs of Collatz-like systems. The formalization in this section is a worked instance of that program.

3.9 Initial configuration

Lemma 3.38 (`initial_config`)**.** The configuration $\mathrm{config}\,[f, f, f]0 =< fff >$ has value 8 and counter 0.

Proof. Compute the left fold: $0 \xrightarrow{<} 1 \xrightarrow{f} 2 \xrightarrow{f} 4 \xrightarrow{f} 8 \xrightarrow{>} 8$. There are no counter symbols. □

4 FAR/CPS window saturation

A *window* of width w over a string s is a length-w factor of the $(w-1)$-padded string $\hat{}^{w-1} s \, \$^{w-1}$, where $\hat{}$ and $\$$ are fresh left/right padding symbols. The FAR/CPS method searches for a regular (finite-window) superset of all reachable configurations: a set W of width-w windows that contains every window of the initial configuration and is closed under the rewriting rules.

Saturation procedure. Starting from $W_0 = \text{grams} < fff > w$, repeatedly apply every rule $\ell \to r$ in every left context of length $w-1$ and every right context of length $w-1$ whose windows are already known to be reachable, and add all windows of the resulting string to the set. More formally, if $u\ell v$ is such that every length-w window of $u\ell v$ lies in the current W, then every length-w window of urv is added to W. The process is repeated until a fixpoint is reached. The implementation is in `antihydra_far.py`, functions `grams`, `left_contexts`, `right_contexts`, and `saturate`.

When the fixpoint is computed, two properties are checked:

- **Control:** the window $2 >$ (ternary digit 2 adjacent to the boundary) is absent from W. This is a genuine invariant: Antihydra's even dynamic rule appends ternary digit 0 and the odd rule appends ternary digit 1, so a ternary ending 2 can never appear.
- **Target:** no string ending in $t >$ (an odd value with empty counter, i.e. a halting configuration) has all its windows in W. If such a string exists, it is a *phantom*: the abstraction admits a halting configuration that is not actually reachable.

Table 1 shows the results for widths 2–6.

Table 1: FAR/CPS window saturation for Antihydra.

w	$\lvert W \rvert$	passes	time	control	target	phantom
2	39	7	< 0.01s	yes	no	$< t >$
3	185	11	0.01s	yes	no	$< ft >$
4	889	18	0.35s	yes	no	$< ftt >$
5	4333	29	16s	yes	no	$< fttt >$
6	21187	37	542s ($\approx$9 min)	yes	no	$< tftt >$

The phantom family. At every width the target fails: the abstraction admits a halting configuration that is not actually reachable. For widths 2–5 the minimal such phantom is

$$< f\, t^{\,w-2} >$$

(with the convention that for $w = 2$ this is $< t >$). Its value is

$$\text{Val}(< f\, t^{\,w-2} >) = 3 \cdot 2^{\,w-2} - 1,$$

an odd number whose binary expansion ends in a run of $w-2$ ones. Such a value commits to $w-2$ consecutive odd macro steps, each decreasing the counter by 1, but the abstraction sees an empty counter at the boundary and cannot compare the length of the odd run with the amount of counter available. These are the -1-*shadow* configurations of Deliverable 4: integers locally indistinguishable from the (negative) fixed point -1, whose odd-run length is the 2-adic budget $\nu_2(a+1)$. At width $w = 6$ a shorter phantom, $< tftt >$, appears; it is admitted by the width-6 window language while the previous phantom $< ftttt >$ is excluded. Thus each increment of w kills some phantoms and admits new ones, and no finite window width can exclude all halting configurations.

5 Pumping obstruction to regular invariants

A FAR-style non-halting proof exhibits a regular set S of configurations that (i) contains every reachable configuration, (ii) is closed under one rewriting step, and (iii) contains no halting configuration. The file `AntihydraSRSObstruction.lean` proves a hard limitation of such proofs for Antihydra when the counter is stored as a contiguous unary block: if S is recognized with p automaton states, then the future dips of the counter are bounded by p.

Let (a_n, b_n) be the state of the deterministic macro orbit after n steps, starting from a binary configuration $\mathrm{config}\, w_0 b_0$ with $a_0 = \mathrm{Val}(< w_0 >) \geq 2$. Define the *future-dip sequence*

$$D(n) = b_n - \min_{m \geq n} b_m. \tag{22}$$

Thus $D(n)$ measures how far the counter falls from its current level before recovering. The main result is:

Proposition 5.1 (Pumping obstruction)**.** Let S be a set of configurations satisfying (i)–(iii) above and suppose S admits unary-counter pumping below p: whenever $\mathrm{config}\, wb \in S$ with $b \geq p$, there exists $b' < p$ such that $\mathrm{config}\, wb' \in S$. Then for every n with $b_n \geq p$,

$$D(n) < p.$$

Equivalently, $b_n < b_m + p$ for all $m \geq n$ whenever $b_n \geq p$.

Before giving the proof, we isolate the necessary orbit arithmetic and the notion of a halting configuration.

5.1 Orbit arithmetic

The following lemmas are proved in `AntihydraSRSObstruction.lean`. They say that the value orbit is additive in the step index, that shifting the initial counter shifts the counter orbit by the same amount, and that the partial iterate macroIter agrees with the deterministic orbits while it has not halted.

Lemma 5.2 (`valOrbit_add`)**.** For all $a_0, n, j \in \mathbb{N}$,

$$\mathrm{valOrbit}\, a_0 n + j = \mathrm{valOrbit}\, \big(\mathrm{valOrbit}\, a_0 n\big) j.$$

Proof. By induction on j, using the definition $\mathrm{valOrbit}\, aj + 1 = \left\lfloor \frac{3}{2}\, \mathrm{valOrbit}\, aj \right\rfloor$. □

Lemma 5.3 (`counterZ_shift`)**.** For all integers b, b' and all $a_0, j \in \mathbb{N}$,

$$\mathrm{counterZ}\, b' a_0 j = \mathrm{counterZ}\, b a_0 j + (b' - b).$$

Proof. Both sides satisfy the same recurrence in j with the same initial value, because counter increments depend only on the value orbit, not on the absolute counter level. □

Lemma 5.4 (`counterZ_add`)**.** For all $b_0 \in \mathbb{Z}$, $a_0, n, j \in \mathbb{N}$,

$$\mathrm{counterZ}\, b_0 a_0 n + j = \mathrm{counterZ}\, \big(\mathrm{counterZ}\, b_0 a_0 n\big)\big(\mathrm{valOrbit}\, a_0 n\big) j.$$

Proof. By induction on j, using Lemma 5.2 and the recurrence for $\mathrm{counterZ}$. □

Lemma 5.5 (`macroIter_eq_orbit`)**.** If $\mathrm{macroIter}(n, (a_0, b_0)) = \mathrm{some}(a_n, b_n)$, then $a_n = \mathrm{valOrbit}\, a_0 n$ and $b_n = \mathrm{counterZ}\, b_0 a_0 n$.

Proof. By induction on n, using the fact that Step applies $a \mapsto \lfloor 3a/2 \rfloor$ on both parity branches and updates the counter by $+2$ or -1 accordingly. □

Lemma 5.6 (`valOrbit_ge_two`)**.** If $a_0 \geq 2$ then $\mathrm{valOrbit}\, a_0 n \geq 2$ for every n.

Proof. The map $a \mapsto \lfloor 3a/2 \rfloor$ preserves the property $a \geq 2$. □

Lemma 5.7 (`macroStep_eq_none_iff`)**.** $\mathrm{Step}(a, b) = \mathrm{none}$ if and only if a is odd and $b = 0$.

Proof. Immediate from the definition of Step. □

Lemma 5.8 (`exists_first_none`)**.** If $\mathrm{macroIter}(N, (a_0, b_0)) = \mathrm{none}$, then there exist j, a_j, b_j such that $\mathrm{macroIter}(j, (a_0, b_0)) = \mathrm{some}(a_j, b_j)$ and $\mathrm{macroIter}(j+1, (a_0, b_0)) = \mathrm{none}$.

Proof. By induction on N. □

Lemma 5.9 (`mem_of_reflTransGen`)**.** If S is closed under one rewriting step and $u \in S$, then every configuration v with $u \to_{\mathcal{R}_A}^* v$ lies in S.

Proof. By induction on the reflexive-transitive closure. □

5.2 Halting configurations

Definition 5.10 (`IsHalting`)**.** A configuration is *halting* if it has the form $\mathrm{config}\, w0$ where w is binary and $\mathrm{Val}(< w >)$ is odd.

Such a configuration is irreducible: the only applicable dynamic rule for an odd value is $t > c \to 1 >$, which requires a counter symbol c; the counter is empty, so no rule fires. By Lemma 5.7 this is exactly the macro halting condition.

5.3 Proof of the pumping obstruction

Proof of Proposition 5.1. Fix n with $b_n \geq p$ and any $m \geq n$. By Theorem 3.32 there is a binary word W_n with $\mathrm{Val}(< W_n >) = a_n$ and a derivation

$$\mathrm{config}\, w_0 b_0 \to_{\mathcal{R}_A}^* \mathrm{config}\, W_n b_n.$$

Since S contains all reachable configurations, $\mathrm{config}\, W_n b_n \in S$. The pumping hypothesis gives $b' < p$ such that $\mathrm{config}\, W_n b' \in S$.

Suppose, for a contradiction, that $b_n \geq b_m + p$. Consider the partial iteration from (a_n, b') for $m - n$ steps. We claim it halts, i.e.

$$\mathrm{macroIter}(m - n, (a_n, b')) = \mathrm{none}\,.$$

If not, suppose it returns $\mathrm{some}(a'', b'')$. By Lemma 5.5,

$$b'' = \mathrm{counterZ}\, b' a_n m - n.$$

Using Lemma 5.3 and Lemma 5.4,

$$\mathrm{counterZ}\, b' a_n m - n = \mathrm{counterZ}\, b_n a_n m - n + (b' - b_n) = b_m + b' - b_n,$$

which is negative because $b_n \geq b_m + p$ and $b' < p$. This is impossible for a natural-number counter, so the iteration must indeed halt.

By Lemma 5.8 there is a first halting transition from (a_n, b'): some j with

$$\mathrm{macroIter}(j, (a_n, b')) = \mathrm{some}(a_j, b_j)$$

and

$$\mathrm{macroIter}(j+1, (a_n, b')) = \mathrm{none}\,.$$

Lemma 5.7 gives that a_j is odd and $b_j = 0$. Apply Theorem 3.32 again, this time starting from $\mathrm{config}\, W_n b'$: there is a binary word W_j with $\mathrm{Val}(< W_j >) = a_j$ (hence odd) and

$$\mathrm{config}\, W_n b' \to^*_{\mathcal{R}_A} \mathrm{config}\, W_j 0.$$

By Lemma 5.9 and closure of S, the halting configuration $\mathrm{config}\, W_j 0$ lies in S, contradicting assumption (iii).

Therefore $b_n < b_m + p$ for every $m \geq n$, i.e. $D(n) < p$. □

5.4 Regular invariants and the corollary

The hypotheses of Proposition 5.1 are bundled in `AntihydraSRSObstruction.lean` into a structure.

Definition 5.11 (`RegularInvariant`)**.** A *regular invariant* with p states for the orbit from $\mathrm{config}\, w_0 b_0$ is a set S of configurations such that:

(reach) every configuration reachable from $\mathrm{config}\, w_0 b_0$ lies in S;

(closed) if $u \in S$ and $u \to_{\mathcal{R}_A} v$, then $v \in S$;

(noHalt) S contains no halting configuration;

(pump) if $\mathrm{config}\, wb \in S$ with $b \geq p$, then there exists $b' < p$ with $\mathrm{config}\, wb' \in S$.

The *orbit counter* is defined by

$$\mathrm{orbitCounter}\, w_0 b_0 n = (\mathrm{macroIter}(n, (\mathrm{Val}(< w_0 >), b_0))).\,\mathrm{getD}((0,0)).2,$$

which reads off b_n while the orbit has not halted. With this notation, Proposition 5.1 becomes:

Theorem 5.12 (`orbitDip_lt`)**.** Let S be a regular invariant with p states for the orbit from $\mathrm{config}\, w_0 b_0$. Then for all n, m with $n \leq m$ and $\mathrm{orbitCounter}\, w_0 b_0 n \geq p$,

$$\mathrm{orbitCounter}\, w_0 b_0 n < \mathrm{orbitCounter}\, w_0 b_0 m + p.$$

Proof. Use Lemma 5.5 to replace $\mathrm{orbitCounter}\, w_0 b_0 n$ by b_n and $\mathrm{orbitCounter}\, w_0 b_0 m$ by b_m, then apply Proposition 5.1. □

In the language of future dips:

Theorem 5.13 (`futureDip_lt_card`)**.** Let S be a regular invariant with p states. Then for every n with $\mathrm{orbitCounter}\, w_0 b_0 n \geq p$,

$$\mathrm{futureDip}(\mathrm{orbitCounter}\, w_0 b_0 \cdot, n) < p.$$

Corollary 5.14 (`no_regular_invariant_of_unbounded_dips`)**.** If the future dips of the orbit counter are unbounded, i.e.

$$\forall q\, \exists n \quad \big(\mathrm{orbitCounter}\, w_0 b_0 n \geq q \;\wedge\; \mathrm{futureDip}(\mathrm{orbitCounter}\, w_0 b_0 \cdot, n) \geq q\big),$$

then no regular invariant of any size exists for the orbit from $\mathrm{config}\, w_0 b_0$.

Proof. A p-state invariant would force

$$\mathrm{futureDip}(\mathrm{orbitCounter}\, w_0 b_0 \cdot, n) < p$$

whenever $\mathrm{orbitCounter}\, w_0 b_0 n \geq p$ (Theorem 5.13), contradicting unboundedness at $q = p$. □

5.5 The pumping property from a finite automaton

The abstract pumping property (pump) in Definition 5.11 is exactly what a DFA recognizer with a contiguous unary counter block provides. We make this literal.

Lemma 5.15 (`exists_iterate_lt_card`)**.** Let σ be a finite type, $g : \sigma \to \sigma$, $q \in \sigma$, and $n \geq |\sigma|$. Then there exists $k < |\sigma|$ such that $g^{[k]}(q) = g^{[n]}(q)$.

Proof. The sequence $g^{[0]}(q), g^{[1]}(q), \ldots, g^{[n]}(q)$ has $n + 1 > |sigma|$ entries, so two of them coincide by the pigeonhole principle. The periodic behaviour of iterates then reduces the larger index to a smaller one. □

Lemma 5.16 (`dfa_pump`)**.** Let M be a DFA over Σ_A with state set σ and let $p = |\sigma|$. If M accepts config wb with $b \geq p$, then M also accepts config wb' for some $b' < p$.

Proof. After reading the value prefix $< w >$, the DFA enters some state q. Reading the counter block c^b iterates the c-transition b times. Since $b \geq p = |\sigma|$, Lemma 5.15 gives $k < p$ with the same final state as b, so M accepts config wk. □

Theorem 5.17 (`regularInvariant_of_dfa`)**.** Let M be a DFA with state set σ whose accepted language $L(M)$ contains every configuration reachable from config $w_0 b_0$, is closed under one $\mathcal{R}_A$-step, and contains no halting configuration. Then $L(M)$ is a regular invariant with $p = |\sigma|$ states.

Proof. Hypotheses (i)–(iii) are exactly reachability, closure, and no-halt. Lemma 5.16 supplies property (pump). □

Corollary 5.18 (`no_dfa_invariant_of_unbounded_dips`)**.** If the future dips of the orbit counter from config $w_0 b_0$ are unbounded, then no DFA recognizes a closed, reachable-containing, halting-free set of configurations. In particular, no FAR-style non-halting proof of Antihydra over unary-counter encodings exists at any number of states.

Proof. Combine Theorem 5.17 with Corollary 5.14. □

5.6 Relation to the FAR/CPS search

The window-saturation experiments of Section 4 produce, for each width w, a regular over-approximation of the reachable configurations. The obstruction of this section says that any genuine regular invariant for Antihydra must bound future dips by the number of automaton states. Since the Antihydra counter performs a random walk with positive drift, future dips are believed to be unbounded; if that is true, then no finite automaton can witness non-halting, and the FAR/CPS method is fundamentally incomplete for this machine. The phantom configurations observed in Table 1 are the concrete symptom: the window abstraction is too weak to remember the counter budget needed to survive long odd runs.

It is worth noting the logical strength of the conclusion. The statement "no finite automaton can witness non-halting" is weaker than the statement "Antihydra does not halt." The pumping obstruction gives FAR witness ⇒ dips bounded, so unbounded dips yields two consequences: (1) the machine does not halt, because halting would cap every future dip by the maximum counter value ever reached; and (2) no FAR-style certificate exists. Thus unbounded dips is a strictly stronger result than either conclusion alone; the FAR incompleteness theorem is the additional meta-mathematical information that one natural class of certificates is too weak to establish the true non-halting behaviour.

References

[1] Emre Yolcu, Scott Aaronson, and Marijn J. H. Heule, *An Automated Approach to the Collatz Conjecture*, arXiv:2105.14697v3 [cs.LO], 2022.

[2] Ralf Stephan (with Claude Code), `AntihydraMachine.lean`: BusyLean machine-checked macro analysis of the Antihydra BB(6) candidate, 2026.

[3] Ralf Stephan (with Claude Code), `AntihydraSRS.lean`: the mixed-base string-rewriting system for Antihydra and its correctness proof, 2026.

[4] Ralf Stephan (with Claude Code), `AntihydraSRSSimulation.lean`: faithful simulation of the Antihydra macro orbit by arbitrary SRS derivations, 2026.

[5] Ralf Stephan (with Claude Code), `AntihydraSRSForward.lean`: constructive forward simulation of the Antihydra macro orbit by an explicit rewriting strategy, 2026.

[6] Ralf Stephan (with Claude Code), `AntihydraSRSObstruction.lean`: pumping obstruction to regular invariants of Antihydra over unary-counter encodings, 2026.

[7] Ralf Stephan (with Claude Code), `AntihydraSRSaxiom.lean`: the BusyLean-free Antihydra macro model used by `AntihydraSRS.lean`, 2026.

[8] Ralf Stephan (with Claude Code), `MixedBaseRepresentation.lean`: mixed-base representations and the base-swap lemma, 2026.

[9] Ralf Stephan (with Claude Code), `Basic.lean`: string rewriting systems, rewrite steps, and termination (Book–Otto §2), 2026.

[10] Ralf Stephan (with Claude Code), *Report B3 — Detailed plans for every research idea in Deliverable 3*, project internal report, §8 "Formal verification track in Lean 4", 2026.

[11] Ralf Stephan (with Claude Code), *3-forward-strategy.html* — Forward-strategy analysis of the YAH paper, project internal report, §5 "Formal verification track", 2026.